

Contents

Linux Basics

SSH

File Permissions

Redirection

Automation Services

System Monitoring

Process

Networking

Ques: 1

- What is the use of echo command?

echo is a command that outputs the strings that are passed to it as arguments.

echo "Hello World!"

Ques: 2

- How to check the computer name or hostname in Linux?

Using 'hostname' command

PathNex
Shaping Careers, Building Futures

Ques: 3

- How to check the name of current user in Linux terminal?

whoami

PathNex
Shaping Careers, Building Futures

Ques: 4

- How to check your current path/directory you are working in?

pwd

PathNex
Shaping Careers, Building Futures

How to see all users in a Linux?

cut -d: -f1 /etc/passwd

PathNex
Shaping Careers, Building Futures

Ques: 5

- Explain the difference between relative and absolute path?

Relative path: Start from current working directory

Absolute path: The full path to a file or directory

PathNex
Shaping Careers, Building Futures

Ques: 6

- Which command to be used to create a file in Linux?

touch

vi

vim

nano

Ques: 7

- How will you edit an existing file on a Linux server?

Vi
Vim
nano

Ques: 8

- How to rename a file in Linux?

Using mv command

PathNex
Shaping Careers, Building Futures

Ques: 9

- How to search for a string in a file?

Using grep command

PathNex
Shaping Careers, Building Futures

Ques: 10

- Difference between grep and egrep?

In egrep, you can search for more than one strings at same time

```
egrep "key1|key2|key4"
```

Ques: 11

- How can you read a file without using cat command?

Using commands
less
more
vi

Ques: 12

- What is the advantage of using less command?

We can easily read big files.

Forward and backward search is easy.

Navigation from top to bottom is easy.

Ques: 13

- How to check a file's permission?

ls -l
ll

getfacl file_name (Not found in MAC)

Ques: 14

- How to check the IP of your Linux server?

ifconfig
ip addr

PathNex
Shaping Careers, Building Futures

Ques: 15

- How to read the top 5 lines in a file?

`head -5 file_name`

PathNex
Shaping Careers, Building Futures

Ques: 16

- How to read the last 5 lines in a file?

`tail -5 file_name`

PathNex
Shaping Careers, Building Futures

Ques: 17

- How to list hidden files?

ls -la

PathNex
Shaping Careers, Building Futures

Ques: 18

- How to see all the recently used commands?

history

PathNex
Shaping Careers, Building Futures

Ques: 19

- What is root?

Admin or super user

/root home directory for root user

/ root directory

Ques: 20

- What is inode and how to find it for a file?

`df -i`

`ls -li`

inode is an index node. It serves as a unique identifier for a specific piece of metadata on a given filesystem

Ques: 21

- Which command can you use for finding files on a Linux system?

using find and locate command

PathNex
Shaping Careers, Building Futures

Ques: 22

- Command for counting words and lines?

WC
WC -l

PathNex
Shaping Careers, Building Futures

Ques: 23

- How can you combine two commands? or What is pipe used for?

We can combine two commands using |

ex: command1 | command2

Pipe is used to combine two commands and redirect output of command 1 to command 2

Ques: 24

- How to view the difference between two files?

```
diff file1 file2
```

PathNex
Shaping Careers, Building Futures

Ques: 25

- What is the use of the shred command ? (permanently delete a file which is unable to recover)

`shred -u file_name`

`shred --remove file_name`

Ques: 26

- How to check system architecture info?

dmidecode and lscpu
(not working in MAC)

PathNex
Shaping Careers, Building Futures

Ques: 27

- How to combine two files?

```
cat file1 file2  
cat file1 file2 > file3
```

Ques: 28

- How can you find the type of file?

file file_name

PathNex
Shaping Careers, Building Futures

Ques: 29

- How to sort the content of a file?

```
sort file_name  
cat file_name | sort
```

PathNex
Shaping Careers, Building Futures

Ques: 30

- Different ways to access a Linux server remotely (from a Windows machine)?

Using some tools and terminal like

putty

git bash

cmd

Ques: 31

- What are different types of permissions for a file in Linux?

Read (r)

Write (w)

Executable (x)

Ques: 32

- Which permission allow a user to run an executable file (script)

We need to provide executable (x) permission to the user.

PathNex
Shaping Careers, Building Futures

Ques: 33

- How to write the output of a command into a file?

command > file_name

usecase – cat test.txt > output.test.txt

PathNex
Shaping Careers, Building Futures

Ques: 34

- How to write something in a file without deleting the existing content?
 - We can append the file using >>
 - `echo "New data entry $(date)" >> datafile.txt`
 - `echo "CPU usage: $(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\[0-9.\]*\\)%*id.*\\/1/" | awk '{print 100 - $1}')" >> cpu_usage.txt`
 - `echo "export PATH=\$PATH:/new/directory" >> ~/.bashrc`

Ques: 35

- How to redirect an error of a command into a file?

To redirect an error we need to use 2>

To redirect both error and output, 2>&1

PathNex
Shaping Careers, Building Futures

Ques: 36

- How to Automate any task or script?

using cron jobs -> A typical cron job entry looks like : `30 2 * * 1 /home/user/backup.sh`

The **at** command is used to schedule one-time tasks to run at a specific time in the future. Unlike cron, which is for recurring tasks, at is used for single, one-time jobs.

For example: at 3pm

Ques: 37

- How to check scheduled jobs?

crontab -l

PathNex
Shaping Careers, Building Futures

Ques: 38

- What is the meaning of this cron job? * * * * *

The cron job expression * * * * * means that the scheduled task will run **every minute** of every hour of every day. Specifically, it breaks down as follows:

The first * (minute) means every minute (0-59).

The second * (hour) means every hour (0-23).

The third * (day of the month) means every day of the month (1-31).

The fourth * (month) means every month (1-12).

The fifth * (day of the week) means every day of the week (0-6), where 0 is Sunday and 6 is Saturday.

In other words, the task will execute **every single minute**, all day, every day, indefinitely.

This is often used for tasks that need to run very frequently, though you might want to be careful with it since it can put a heavy load on your system if not managed properly.

Ques: 39

- If your cron job didn't work, how would you check?

check system time

crontab entry

check /var/log/messages

PathNex
Shaping Careers, Building Futures

Ques: 40

- What is daemon service?

Service that keep running in background.

Example: httpd, sshd, chronyd

PathNex
Shaping Careers, Building Futures

Ques: 41

- How to check if a service is running or not?

`systemctl status service_name`

PathNex
Shaping Careers, Building Futures

Ques: 42

- How to start/stop any service?

```
systemctl start service_name  
systemctl stop service_name
```

PathNex
Shaping Careers, Building Futures

Ques: 43

- How to check for free disk space?

We can use df command

Usecase -> `df -h`

Helps us to see the mounted disks.

Ques: 44

- How to check the size of a directory's content?

We can use “**du**” command

PathNex
Shaping Careers, Building Futures

Ques: 45

- How to check CPU usage for a process?

We can use **top** command

PathNex
Shaping Careers, Building Futures

Ques: 46

- What is a process in Linux?

An instance of a running program. Whenever you start a program/application or execute a command, a process is created.

For every process a unique no. is assigned which is called PID
(Process ID)

Ques: 47

- How to check if a process/application is running or not?

Using ps command Usecase → `ps aux | grep slack`

- 1.a:** Lists processes of all users, not just the current user. Normally, `ps` shows processes belonging to the current user only, but with `a`, it includes processes from all users.
- 2.u:** Displays the processes in a user-oriented format, showing information such as the user who started the process, CPU and memory usage, and the start time of the process.
- 3.x:** Includes processes that don't have a controlling terminal (such as background processes and system services). Normally, `ps` only shows processes running in a terminal session, but `x` will show all processes.

Ques: 48

- How to terminate/stop a running process?

Using kill command Usecase → `kill -9 76569`

- 9: This is the signal number (SIGKILL), which forcefully kills a process. The -9 signal cannot be caught or ignored by the process, so it guarantees that the process is terminated immediately.

Ques: 49

- How to check if a IP/Server is accessible or not?

We can use ping or telnet command

PathNex
Shaping Careers, Building Futures

Ques: 50

- Which command to use to get info about ports?

We can use **netstat** command

PathNex
Shaping Careers, Building Futures

Ques: 51

- How to check open port on Linux system?

netstat -tuln | grep port_no

Usecase → **netstat -tuln | grep 80**

-t : Displays TCP ports.

-u : Displays UDP ports.

-l : Lists only listening ports.

-n : Shows numeric addresses (avoids resolving hostnames).

Ques: 52

- **Common Use Cases for Lsof**

Identify Which Processes Are Using a Specific File - You can use Lsof to check which process has a particular file open. This is useful when you need to know which application or service is currently accessing or locking a file.

Example: `Lsof /path/to/file` - This command will show all processes that have `/path/to/file` open.

Finding Open Files by a Specific Process - You can list all open files for a specific process by using its PID (Process ID).

Example: `Lsof -p <PID>` This shows all open files by the process with ID `<PID>`.

Identify Which Process Is Using a Specific Port - This is very useful for troubleshooting network issues or checking which service is listening on a specific port. For instance, if you're unsure which process is using port 80 (HTTP), you can run:

Example: `Lsof -i :80 ->` This will show all processes that are listening on port 80.

Ques: 53

- How to check network interfaces in Linux?

We can use **ifconfig** and **netstat** command

PathNex
Shaping Careers, Building Futures

Ques: 54

- Difference between Telnet and SSH?

SSH is secured
TELNET is not secured.

PathNex
Shaping Careers, Building Futures

Ques: 55

- Which service should be running on server to allow you to connect remotely?

ssh or sshd

PathNex
Shaping Careers, Building Futures

Ques: 56

- What is SSH?

SSH or Secure Shell is a network communication protocol that enables two computers/devices to communicate and share data.

PathNex
Shaping Careers, Building Futures

Ques: 57

- Why SSH called as Secure Shell?

The communication between host and client will be in encrypted format.

PathNex
Shaping Careers, Building Futures

Ques: 58

- Which command is used to access a linux system from a terminal or another linux server?

`ssh user@198.168.x.x`

PathNex
Shaping Careers, Building Futures

Run Levels

Runlevel	Description
0	Shutdown: The system is halted and powered off.
1	Single-user mode: This is for system maintenance, typically used for recovery or repair. Only the root user can log in, and most services are not started.
2	Multi-user mode (without networking): A multi-user state where no networking services are started (or minimal network functionality).
3	Multi-user mode with networking: A multi-user state with full networking enabled, but typically no graphical user interface (GUI). This is common for server environments.
4	Unused/Custom: Traditionally reserved for custom configurations or user-specific states, but it is not typically used by modern distributions.
5	Multi-user mode with GUI: A multi-user state with networking and a graphical user interface (GUI) enabled. This is typically the default runlevel for desktop systems.
6	Reboot: The system will reboot.

Linux Shell Special Variables

- **\$0** → Script name
- **\$1, \$2, \$3 ...** → Command-line arguments
 - **\$1** = first argument
 - **\$2** = second argument

Example:

```
./script.sh file1 file2
```

```
$1 → file1
```

```
$2 → file2
```

- **\$#** → Total number of arguments
- **\$@** → All arguments (separately)
- **\$*** → All arguments (single string)
- **\$?** → Exit status of last command
 - **0** = Success
 - **Non-zero** = Failure

What is rsync?

A tool to **sync** files between locations, transferring only **changed parts**.

Syntax

rsync [options] SRC DEST

Key Options

- a: **Archive** (preserve attributes)
- v: **Verbose** (details)
- z: **Compress**
- r: **Recursive** (for dirs)
- u: **Update** (skip newer files)
- n: **Dry-run**
- e: **Remote shell** (SSH)
- delete: **Delete** in DEST not in SRC
- P: **Progress** & partial transfers

Examples

Local to local:

```
rsync -av /src/ /dst/
```

Local to remote:

```
rsync -avz /src/ user@remote:/dst/
```

Dry-run:

```
rsync -av --dry-run /src/ /dst/
```